Classification Assignment

Problem Statement or Requirement:

A requirement from the Hospital, Management asked us to create a predictive model which will predict the Chronic Kidney Disease (CKD) based on the several parameters. The Client has provided the dataset of the same.

1.) Identify your problem statement

- The problem statement is to predict chronic kidney disease based on various parameters from the dataset.
- The given dataset is numerical and output are give so this is a **Supervised Binary Classification problem.**

2.) Basic info about the dataset

- This dataset is named as CKD and has 25 columns and 399 rows.

3.) Mention the pre-processing method

- The dataset has few columns with nominal categorical data.
- **One-Hot encoding** is used to convert categorical data to numerical data.
- **Standard Scaler** is used for preprocessing the inputs for the model.

4.) Develop a good model with good evaluation metric.

- The data is to be trained in various ML classification models using **GridSearchCV.**
- The models will be evaluated with confusion matric, classification report and roc_auc_score and best model will be deployed.

5.) Algorithm Results:

- **LOGISTIC REGRESSION**

```
# Best gridseach parameter
grid.best_params_
```

```
{'C': 0.01, 'penalty': 'l2', 'solver': 'lbfgs'}
```

```
# Evaluation metrics
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test,y_pred)
print(cm)
```

```
[[36  0]
 [ 1 63]]
```

```
# Classification report
from sklearn.metrics import classification_report
report = classification_report(y_test,y_pred)
print(report)
```

```
              precision    recall  f1-score   support

           0       0.97      1.00      0.99        36
           1       1.00      0.98      0.99        64

    accuracy                           0.99       100
   macro avg       0.99      0.99      0.99       100
weighted avg       0.99      0.99      0.99       100
```

```
from sklearn.metrics import roc_auc_score
roc_auc_score(y_test,grid.predict_proba(x_test)[:,1])
```

```
np.float64(1.0)
```

- **DECISION TREE**

```
# Best gridseach parameter
grid.best_params_
```

```
{'criterion': 'entropy', 'max_features': 'log2', 'splitter': 'best'}
```

```
# Evaluation metrics
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test,y_pred)
print(cm)
```

```
[[34  2]
 [ 4 60]]
```

```
# Classification report
from sklearn.metrics import classification_report
report = classification_report(y_test,y_pred)
print(report)
```

```
              precision    recall  f1-score   support

           0       0.89      0.94      0.92        36
           1       0.97      0.94      0.95        64

    accuracy                           0.94       100
   macro avg       0.93      0.94      0.94       100
weighted avg       0.94      0.94      0.94       100
```

```
from sklearn.metrics import roc_auc_score
roc_auc_score(y_test,grid.predict_proba(x_test)[:,1])
```

```
np.float64(0.9409722222222222)
```

- **RANDOM FOREST**

```
# Best gridseach parameter
grid.best_params_
```

```
{'class_weight': 'balanced',
 'criterion': 'log_loss',
 'max_features': 'sqrt',
 'n_estimators': 50}
```

```
# Evaluation metrics
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test,y_pred)
print(cm)
```

```
[[35  1]
 [ 0 64]]
```

```
# Classification report
from sklearn.metrics import classification_report
report = classification_report(y_test,y_pred)
print(report)
```

```
              precision    recall  f1-score   support

           0       1.00      0.97      0.99        36
           1       0.98      1.00      0.99        64

    accuracy                           0.99       100
   macro avg       0.99      0.99      0.99       100
weighted avg       0.99      0.99      0.99       100
```

```
from sklearn.metrics import roc_auc_score
roc_auc_score(y_test,grid.predict_proba(x_test)[:,1])
```

```
np.float64(1.0)
```

- **SUPPORT VECTOR CLASSIFIER**

```
# Best gridseach parameter
grid.best_params_
```

```
{'C': 1, 'gamma': 'scale', 'kernel': 'rbf'}
```

```
# Evaluation metrics
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test,y_pred)
print(cm)
```

```
[[36  0]
 [ 1 63]]
```

```
# Classification report
from sklearn.metrics import classification_report
report = classification_report(y_test,y_pred)
print(report)
```

```
              precision    recall  f1-score   support

           0       0.97      1.00      0.99        36
           1       1.00      0.98      0.99        64

    accuracy                           0.99       100
   macro avg       0.99      0.99      0.99       100
weighted avg       0.99      0.99      0.99       100
```

```
from sklearn.metrics import roc_auc_score
roc_auc_score(y_test,grid.predict_proba(x_test)[:,1])
```

```
np.float64(1.0)
```

- **KNEAREST NEIGHBOURS**

```
# Best gridseach parameter
grid.best_params_
```

✓ 0.0s

```
{'algorithm': 'auto', 'n_neighbors': 3, 'weights': 'uniform'}
```

```
# Evaluation metrics
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test,y_pred)
print(cm)
```

✓ 0.0s

```
[[36  0]
 [ 3 61]]
```

```
# Classification report
from sklearn.metrics import classification_report
report = classification_report(y_test,y_pred)
print(report)
```

✓ 0.0s

```
              precision    recall  f1-score   support

           0       0.92      1.00      0.96        36
           1       1.00      0.95      0.98        64

    accuracy                           0.97       100
   macro avg       0.96      0.98      0.97       100
weighted avg       0.97      0.97      0.97       100
```

```
from sklearn.metrics import roc_auc_score
roc_auc_score(y_test,grid.predict_proba(x_test)[:,1])
```

✓ 0.0s

```
np.float64(0.9993489583333334)
```

6.) **RESULTS:**

- After training the models with different classification algorithms and evaluating them SVM and Logistic Regression models have had higher accuracy, precision, recall, f1 score and roc_auc_score.
- Therefore, **Logistic regression and SVM are best models for deployment.**